

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – High (Coding Challenge)

COURSE CODE: CSA09

COURSE NAME: Programming in Java

COURSE OUTCOME COVERED

CO1: *Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)*

QUESTIONS: 10

Total Marks: 100

ANNEXURE A Coding Challenge

Code of Conduct:

I K.Amrutha Sandhya(192524270) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Amrutha

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Coding Challenge

1. Library Book Management using Classes & Encapsulation

Design a Book class with private fields such as bookId, title, author, and price. Implement constructors, getters, setters, and methods to issue and return books. Create a menu-driven program to manage multiple books using an array of objects. Also, construct suitable test cases to validate book addition, issue, return, and search operations. BTL: BL3 (Apply)

```
import java.util.Scanner;
```

CODE:

```
class Book {  
    private int bookId;  
    private String title;  
    private String author;  
    private double price;  
    private boolean issued;  
    public Book(int bookId, String title, String author, double price) {  
        this.bookId = bookId;  
        this.title = title;  
        this.author = author;  
        this.price = price;  
        this.issued = false;  
    }  
    public int getBookId() {  
        return bookId;  
    }  
    public String getTitle() {  
        return title;  
    }  
    public boolean isIssued() {  
        return issued;  
    }  
    public void issueBook() {  
        if (!issued) {  
            issued = true;  
            System.out.println("Book issued successfully.");  
        } else {
```

```

        System.out.println("Book is already issued.");
    }
}

public void returnBook() {
    if (issued) {
        issued = false;
        System.out.println("Book returned successfully.");
    } else {
        System.out.println("Book was not issued.");
    }
}

public void display() {
    System.out.println("\nBook ID : " + bookId);
    System.out.println("Title : " + title);
    System.out.println("Author : " + author);
    System.out.println("Price : " + price);
    System.out.println("Status : " + (issued ? "Issued" : "Available"));
}
}

public class LibraryManagement {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Book[] books = new Book[10];
        int count = 0;
        int choice;
        do {
            System.out.println("\n===== Library Menu =====");
            System.out.println("1. Add Book");
            System.out.println("2. Display Books");
            System.out.println("3. Issue Book");
            System.out.println("4. Return Book");
            System.out.println("5. Search Book");
            System.out.println("6. Exit");
            System.out.print("Enter Choice: ");
            choice = sc.nextInt();
            switch (choice) {

```

case 1:

```
System.out.print("Enter Book ID: ");
int id = sc.nextInt();
sc.nextLine();
System.out.print("Enter Title: ");
String title = sc.nextLine();
System.out.print("Enter Author: ");
String author = sc.nextLine();
System.out.print("Enter Price: ");
double price = sc.nextDouble();
books[count++] = new Book(id, title, author, price);
System.out.println("Book Added Successfully.");
break;
```

case 2:

```
if (count == 0) {
    System.out.println("No Books Available.");
} else {
    for (int i = 0; i < count; i++) {
        books[i].display();
    }
}
break;
```

case 3:

```
System.out.print("Enter Book ID to Issue: ");
id = sc.nextInt();
boolean found = false;
for (int i = 0; i < count; i++) {
    if (books[i].getBookId() == id) {
        books[i].issueBook();
        found = true;
        break;
    }
}
if (!found)
    System.out.println("Book Not Found.");
break;
```

case 4:

```

        System.out.print("Enter Book ID to Return: ");

        id = sc.nextInt();

        found = false;

        for (int i = 0; i < count; i++) {

            if (books[i].getBookId() == id) {

                books[i].returnBook();

                found = true;

                break;

            }

        }

        if (!found)

            System.out.println("Book Not Found.");

        break;

    case 5:

        System.out.print("Enter Book ID to Search: ");

        id = sc.nextInt();

        found = false;

        for (int i = 0; i < count; i++) {

            if (books[i].getBookId() == id) {

                books[i].display();

                found = true;

                break;

            }

        }

        if (!found)

            System.out.println("Book Not Found.");

        break;

    case 6:

        System.out.println("Thank You!");

        break;

    default:

        System.out.println("Invalid Choice.");

    }

} while (choice != 6);

sc.close();

}

}

```

OUTPUT:

```
public class LibraryManagement {
    public static void main(String[] args) {
        do {
            System.out.println("3. Return Book ");
            System.out.println("4. Exit");
            System.out.print("Enter Choice: ");
            choice = sc.nextInt();

            switch (choice) {
                case 1:
                    b.display();
                    break;
                case 2:
                    b.issueBook();
                    break;
                case 3:
                    b.returnBook();
                    break;
                case 4:
                    System.out.println("Thank You!");
                    break;
                default:
                    System.out.println("Invalid Choice!");
            }

        } while (choice != 4);

        sc.close();
    }
}
```

Enter Book ID: 1
Enter Title: life without limits
Enter Author: nick
Enter Price: 999

1. Display
2. Issue Book
3. Return Book
4. Exit
Enter Choice: 1

Book ID : 1
Title : life without limits
Author : nick
Price : 999.0
Status : Available

1. Display
2. Issue Book
3. Return Book
4. Exit
Enter Choice: 2
Book Issued Successfully.

1. Display
2. Issue Book

2. Employee Payroll System using Inheritance

Design a payroll system with a base class Employee and derived classes PermanentEmployee and ContractEmployee. Override a method calculateSalary() in each subclass to compute salary differently. Display employee details and salary using runtime polymorphism. Also, construct suitable test cases to validate salary calculation for different employee types. BTL: BL3 (Apply)

CODE:

```
import java.util.Scanner;

class Employee {
    String name;
    int id;

    Employee(String name, int id) {
        this.name = name;
        this.id = id;
    }

    void calculateSalary() {
        System.out.println("Salary Calculation");
    }

    void display() {
        System.out.println("Employee ID : " + id);
        System.out.println("Employee Name : " + name);
    }
}

class PermanentEmployee extends Employee {
    double basicSalary, bonus;
```

```

PermanentEmployee(String name, int id, double basicSalary, double bonus) {
    super(name, id);
    this.basicSalary = basicSalary;
    this.bonus = bonus;
}

void calculateSalary() {
    System.out.println("Salary : " + (basicSalary + bonus));
}
}

class ContractEmployee extends Employee {
    double salaryPerDay;
    int days;

    ContractEmployee(String name, int id, double salaryPerDay, int days) {
        super(name, id);
        this.salaryPerDay = salaryPerDay;
        this.days = days;
    }

    void calculateSalary() {
        System.out.println("Salary : " + (salaryPerDay * days));
    }
}

public class PayrollSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("1. Permanent Employee");
        System.out.println("2. Contract Employee");
        System.out.print("Enter Choice: ");
        int choice = sc.nextInt();
        sc.nextLine();

        Employee emp;
        if (choice == 1) {
            System.out.print("Enter Employee ID: ");
            int id = sc.nextInt();
            sc.nextLine();

            System.out.print("Enter Employee Name: ");
            String name = sc.nextLine();

            System.out.print("Enter Basic Salary: ");

```

```

        double basic = sc.nextDouble();

        System.out.print("Enter Bonus: ");

        double bonus = sc.nextDouble();

        emp = new PermanentEmployee(name, id, basic, bonus);
    } else {

        System.out.print("Enter Employee ID: ");

        int id = sc.nextInt();

        sc.nextLine();

        System.out.print("Enter Employee Name: ");

        String name = sc.nextLine();

        System.out.print("Enter Salary Per Day: ");

        double salary = sc.nextDouble();

        System.out.print("Enter Number of Days: ");

        int days = sc.nextInt();

        emp = new ContractEmployee(name, id, salary, days);
    }

    System.out.println("\nEmployee Details");

    emp.display();

    emp.calculateSalary();

    sc.close();

}

}

```

OUTPUT:

The screenshot shows a Java IDE with two panes. The left pane displays the source code for a payroll system, and the right pane shows the program's output.

Source Code:

```

class PermanentEmployee extends Employee {
    PermanentEmployee(String name, int id, double basicSalary, double bonus) {
        this.basicSalary = basicSalary;
        this.bonus = bonus;
    }

    void calculateSalary() {
        System.out.println("Salary : " + (basicSalary + bonus));
    }
}

class ContractEmployee extends Employee {
    double salaryPerDay;
    int days;

    ContractEmployee(String name, int id, double salaryPerDay, int days) {
        super(name, id);
        this.salaryPerDay = salaryPerDay;
        this.days = days;
    }

    void calculateSalary() {
        System.out.println("Salary : " + (salaryPerDay * days));
    }
}

public class PayrollSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

```

Output:

```

1. Permanent Employee
2. Contract Employee
Enter Choice: 1
Enter Employee ID: 124
Enter Employee Name: amrutha
Enter Basic Salary: 999999
Enter Bonus: 99999

Employee Details
Employee ID : 124
Employee Name : amrutha
Salary : 1099998.0

```


3. Vehicle Rental System using Runtime Polymorphism

Develop a vehicle rental system with a superclass Vehicle and subclasses Car, Bike, and Bus.

Override the method calculateRent(int days) in each subclass. Store vehicles in an array of Vehicle references and generate rental bills dynamically. Also, construct suitable test cases to validate rent calculation for different vehicle categories. BTL: BL4 (Analyze)

CODE:

```
import java.util.Scanner;

class Vehicle {
    void calculateRent(int days) {
        System.out.println("Rent Calculation");
    }
}

class Car extends Vehicle {
    void calculateRent(int days) {
        System.out.println("Car Rent = ₹" + (days * 1000));
    }
}

class Bike extends Vehicle {
    void calculateRent(int days) {
        System.out.println("Bike Rent = ₹" + (days * 500));
    }
}

class Bus extends Vehicle {
    void calculateRent(int days) {
        System.out.println("Bus Rent = ₹" + (days * 3000));
    }
}

public class VehicleRental {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("1. Car");
        System.out.println("2. Bike");
        System.out.println("3. Bus");

        System.out.print("Enter Vehicle Choice: ");

        int choice = sc.nextInt();
```

```

        System.out.print("Enter Number of Days: ");

        int days = sc.nextInt();

        Vehicle v;

        switch (choice) {

            case 1:

                v = new Car();

                break;

            case 2:

                v = new Bike();

                break;

            case 3:

                v = new Bus();

                break;

            default:

                System.out.println("Invalid Choice");

                sc.close();

                return;

        }

        v.calculateRent(days);

        sc.close();

    }

}

```

OUTPUT:

<pre> public class VehicleRental { public static void main(String[] args) { int choice = sc.nextInt(); System.out.print("Enter Number of Days: "); int days = sc.nextInt(); Vehicle v; switch (choice) { case 1: v = new Car(); break; case 2: v = new Bike(); break; case 3: v = new Bus(); break; default: System.out.println("Invalid Choice"); sc.close(); return; } v.calculateRent(days); sc.close(); } } </pre>	<pre> 1. Car 2. Bike 3. Bus Enter Vehicle Choice: 1 Enter Number of Days: 80 Car Rent = ₹80000 </pre>
---	---

4. Bank Account System using Data Hiding

Design a BankAccount class with private fields accountNumber, holderName, and balance.

Implement methods for deposit, withdrawal, and balance inquiry with proper validation.

Prevent direct modification of balance from outside the class. Also, construct suitable test cases to validate valid and invalid transactions. BTL: BL3 (Apply)

CODE:

```
import java.util.Scanner;

class BankAccount {
    private int accountNumber;
    private String holderName;
    private double balance;

    BankAccount(int accountNumber, String holderName, double balance) {
        this.accountNumber = accountNumber;
        this.holderName = holderName;
        this.balance = balance;
    }

    void deposit(double amount) {
        balance = balance + amount;
        System.out.println("Amount Deposited Successfully.");
    }

    void withdraw(double amount) {
        if (amount <= balance) {
            balance = balance - amount;
            System.out.println("Amount Withdrawn Successfully.");
        } else {
            System.out.println("Insufficient Balance.");
        }
    }

    void displayBalance() {
        System.out.println("Account Number : " + accountNumber);
        System.out.println("Account Holder : " + holderName);
        System.out.println("Balance : " + balance);
    }
}

public class BankDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```

System.out.print("Enter Account Number: ");

int accNo = sc.nextInt();

sc.nextLine();

System.out.print("Enter Account Holder Name: ");

String name = sc.nextLine();

System.out.print("Enter Initial Balance: ");

double balance = sc.nextDouble();

BankAccount b = new BankAccount(accNo, name, balance);

System.out.println("\n1. Deposit");

System.out.println("2. Withdraw");

System.out.println("3. Display Balance");

System.out.print("Enter Choice: ");

int choice = sc.nextInt();

switch (choice) {

    case 1:

        System.out.print("Enter Deposit Amount: ");

        double dep = sc.nextDouble();

        b.deposit(dep);

        b.displayBalance();

        break;

    case 2:

        System.out.print("Enter Withdraw Amount: ");

        double wd = sc.nextDouble();

        b.withdraw(wd);

        b.displayBalance();

        break;

    case 3:

        b.displayBalance();

        break;

    default:

        System.out.println("Invalid Choice");

}

sc.close();

}

}

```

OUTPUT:

```
public class BankDemo {
    public static void main(String[] args) {
        int choice = sc.nextInt();

        switch (choice) {
            case 1:
                System.out.print("Enter Deposit Amount: ");
                double dep = sc.nextDouble();
                b.deposit(dep);
                b.displayBalance();
                break;

            case 2:
                System.out.print("Enter Withdraw Amount: ");
                double wd = sc.nextDouble();
                b.withdraw(wd);
                b.displayBalance();
                break;

            case 3:
                b.displayBalance();
                break;

            default:
                System.out.println("Invalid Choice");
        }

        sc.close();
    }
}
```

Enter Account Number: 2926
Enter Account Holder Name: Amrutha Sandhya
Enter Initial Balance: 1000

1. Deposit
2. Withdraw
3. Display Balance
Enter Choice: 3
Account Number : 2926
Account Holder : Amrutha Sandhya
Balance : 1000.0

5. University Admission System using Method Overloading

Create an Admission class that overloads the method calculateFee() for:

- ☐ Undergraduate students,
- ☐ Postgraduate students,
- ☐ Scholarship students.

Display the fee structure based on user input and compare the overloaded method executions.

Also, construct suitable test cases to validate each overloaded version. BTL: BL3 (Apply)

CODE:

```
import java.util.Scanner;

class Admission {
    // Undergraduate Fee
    void calculateFee(double fee) {
        System.out.println("Undergraduate Fee: ₹" + fee);
    }

    // Postgraduate Fee
    void calculateFee(double fee, int year) {
        System.out.println("Postgraduate Fee: ₹" + fee);
    }

    // Scholarship Student Fee
    void calculateFee(double fee, double scholarship) {
        double finalFee = fee - scholarship;
        System.out.println("Scholarship Fee: ₹" + finalFee);
    }
}

public class AdmissionDemo {
```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    Admission a = new Admission();
    System.out.println("1. Undergraduate");
    System.out.println("2. Postgraduate");
    System.out.println("3. Scholarship Student");
    System.out.print("Enter Choice: ");
    int choice = sc.nextInt();
    switch (choice) {
        case 1:
            System.out.print("Enter Undergraduate Fee: ");
            double ugFee = sc.nextDouble();
            a.calculateFee(ugFee);
            break;
        case 2:
            System.out.print("Enter Postgraduate Fee: ");
            double pgFee = sc.nextDouble();
            a.calculateFee(pgFee, 2);
            break;
        case 3:
            System.out.print("Enter Total Fee: ");
            double fee = sc.nextDouble();
            System.out.print("Enter Scholarship Amount: ");
            double scholarship = sc.nextDouble();
            a.calculateFee(fee, scholarship);
            break;
        default:
            System.out.println("Invalid Choice");
    }
    sc.close();
}
}

```

OUTPUT:

```
public class AdmissionDemo {
    public static void main(String[] args) {
        switch (choice) {
            case 1:
                a.calculateFee(ugFee);
                break;

            case 2:
                System.out.print("Enter Postgraduate Fee: ");
                double pgFee = sc.nextDouble();
                a.calculateFee(pgFee, 2);
                break;

            case 3:
                System.out.print("Enter Total Fee: ");
                double fee = sc.nextDouble();

                System.out.print("Enter Scholarship Amount: ");
                double scholarship = sc.nextDouble();

                a.calculateFee(fee, scholarship);
                break;

            default:
                System.out.println("Invalid Choice");
        }
        sc.close();
    }
}
```

```
1. Undergraduate
2. Postgraduate
3. Scholarship Student
Enter Choice: 3
Enter Total Fee: 2373473
Enter Scholarship Amount: 374638
Scholarship Fee: ₹1998835.0
```

6. Shape Area Calculator using Abstract Class & Polymorphism

Design an abstract class Shape with an abstract method area(). Implement subclasses Circle, Rectangle, and Triangle. Use an array of Shape references to compute and display areas dynamically. Also, construct suitable test cases to validate area calculations and polymorphic behavior. BTL: BL4 (Analyze)

CODE:

```
import java.util.Scanner;

// Abstract Class
abstract class Shape {
    abstract void area();
}

// Circle Class
class Circle extends Shape {
    double radius;

    Circle(double radius) {
        this.radius = radius;
    }

    void area() {
        System.out.println("Area of Circle = " + (3.14 * radius * radius));
    }
}

// Rectangle Class
class Rectangle extends Shape {
    double length, breadth;

    Rectangle(double length, double breadth) {
        this.length = length;
    }
}
```

```

        this.breadth = breadth;
    }
    void area() {
        System.out.println("Area of Rectangle = " + (length * breadth));
    }
}

// Triangle Class
class Triangle extends Shape {
    double base, height;
    Triangle(double base, double height) {
        this.base = base;
        this.height = height;
    }
    void area() {
        System.out.println("Area of Triangle = " + (0.5 * base * height));
    }
}

public class ShapeDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Shape s;
        System.out.println("1. Circle");
        System.out.println("2. Rectangle");
        System.out.println("3. Triangle");
        System.out.print("Enter Choice: ");
        int choice = sc.nextInt();
        switch (choice) {
            case 1:
                System.out.print("Enter Radius: ");
                double r = sc.nextDouble();
                s = new Circle(r);
                s.area();
                break;
            case 2:
                System.out.print("Enter Length: ");
                double l = sc.nextDouble();
                System.out.print("Enter Breadth: ");

```



```

        double b = sc.nextDouble();

        s = new Rectangle(l, b);

        s.area();

        break;

    case 3:

        System.out.print("Enter Base: ");

        double base = sc.nextDouble();

        System.out.print("Enter Height: ");

        double h = sc.nextDouble();

        s = new Triangle(base, h);

        s.area();

        break;

    default:

        System.out.println("Invalid Choice");

    }

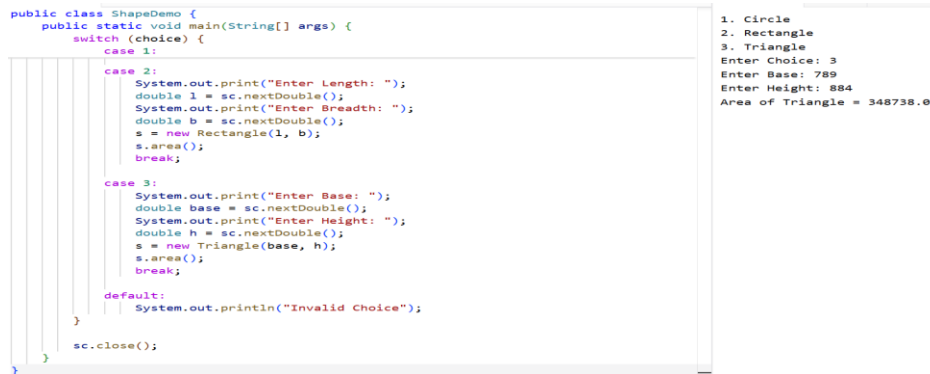
    sc.close();

}

}

```

OUTPUT:



```

public class ShapeDemo {
    public static void main(String[] args) {
        switch (choice) {
            case 1:
            case 2:
                System.out.print("Enter Length: ");
                double l = sc.nextDouble();
                System.out.print("Enter Breadth: ");
                double b = sc.nextDouble();
                s = new Rectangle(l, b);
                s.area();
                break;
            case 3:
                System.out.print("Enter Base: ");
                double base = sc.nextDouble();
                System.out.print("Enter Height: ");
                double h = sc.nextDouble();
                s = new Triangle(base, h);
                s.area();
                break;
            default:
                System.out.println("Invalid Choice");
        }
        sc.close();
    }
}

```

```

1. Circle
2. Rectangle
3. Triangle
Enter Choice: 3
Enter Base: 789
Enter Height: 884
Area of Triangle = 348738.0

```

7. Hospital Management System using Interfaces

Develop a hospital management system with an interface MedicalRecord containing methods addRecord() and displayRecord(). Implement the interface in classes Patient and Doctor.

Demonstrate interface-based abstraction and object interaction. Also, construct suitable test cases to validate record creation and display. BTL: BL3 (Apply)

CODE:

```

import java.util.Scanner;

// Interface

interface MedicalRecord {

    void addRecord();

```

```

        void displayRecord();
    }

// Patient Class

class Patient implements MedicalRecord {
    int patientId;
    String patientName;
    public void addRecord() {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Patient ID: ");
        patientId = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Patient Name: ");
        patientName = sc.nextLine();
    }
    public void displayRecord() {
        System.out.println("\nPatient Record");
        System.out.println("Patient ID : " + patientId);
        System.out.println("Patient Name : " + patientName);
    }
}

// Doctor Class

class Doctor implements MedicalRecord {
    int doctorId;
    String doctorName;
    public void addRecord() {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Doctor ID: ");
        doctorId = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Doctor Name: ");
        doctorName = sc.nextLine();
    }
    public void displayRecord() {
        System.out.println("\nDoctor Record");
        System.out.println("Doctor ID : " + doctorId);
        System.out.println("Doctor Name : " + doctorName);
    }
}

```

```

    }
}

// Main Class

public class HospitalDemo {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("1. Patient");

        System.out.println("2. Doctor");

        System.out.print("Enter Choice: ");

        int choice = sc.nextInt();

        if (choice == 1) {

            Patient p = new Patient();

            p.addRecord();

            p.displayRecord();

        } else if (choice == 2) {

            Doctor d = new Doctor();

            d.addRecord();

            d.displayRecord();

        } else {

            System.out.println("Invalid Choice");

        }

        sc.close();

    }

}

```

OUTPUT:



The screenshot shows a code editor with the Java code on the left and its output on the right. The code is the same as provided in the previous block. The output on the right shows the execution results:

```

1. Patient
2. Doctor
Enter Choice: 2
Enter Doctor ID: 56
Enter Doctor Name: Sai Krishna

Doctor Record
Doctor ID : 56
Doctor Name : Sai Krishna

```

8. Online Shopping Order System using Packages

Create two packages:

- shopping.product
- shopping.order

Implement classes Product and Order in separate packages. Import the packages into a main application that places orders and calculates the total amount. Also, construct suitable test cases to validate package usage, order placement, and bill generation. BTL: BL3 (Apply)

CODE:

```
import java.util.Scanner;

class Product {
    String name;
    double price;
    Product(String name, double price) {
        this.name = name;
        this.price = price;
    }
}

class Order {
    void placeOrder(Product p, int qty) {
        System.out.println("\nProduct Name : " + p.name);
        System.out.println("Price : " + p.price);
        System.out.println("Quantity : " + qty);
        System.out.println("Total Amount : " + (p.price * qty));
    }
}

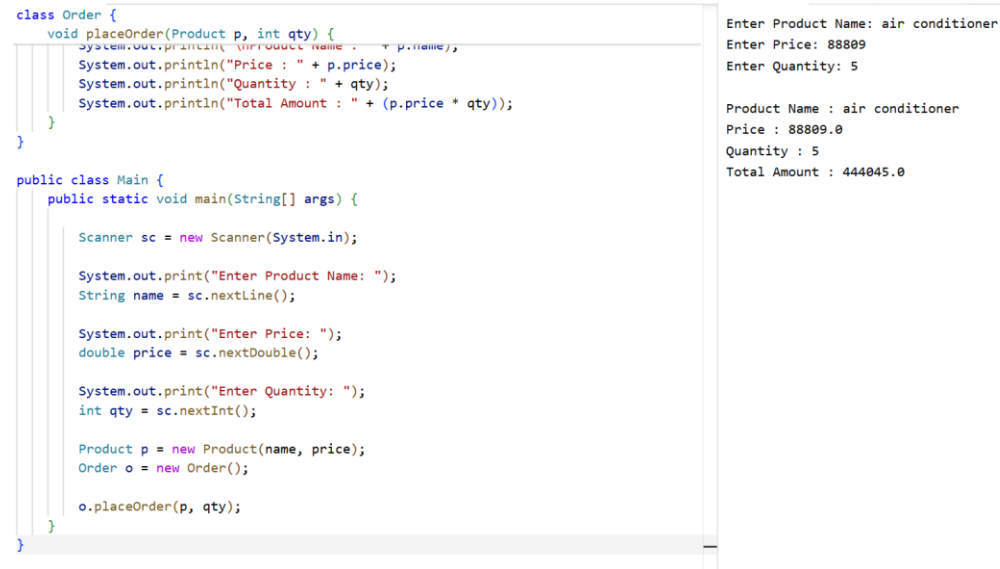
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Product Name: ");
        String name = sc.nextLine();
        System.out.print("Enter Price: ");
        double price = sc.nextDouble();
        System.out.print("Enter Quantity: ");
        int qty = sc.nextInt();
        Product p = new Product(name, price);
        Order o = new Order();
```

```

        o.placeOrder(p, qty);
    }
}

```

OUTPUT:



```

class Order {
    void placeOrder(Product p, int qty) {
        System.out.println("Product Name : " + p.name);
        System.out.println("Price : " + p.price);
        System.out.println("Quantity : " + qty);
        System.out.println("Total Amount : " + (p.price * qty));
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Product Name: ");
        String name = sc.nextLine();

        System.out.print("Enter Price: ");
        double price = sc.nextDouble();

        System.out.print("Enter Quantity: ");
        int qty = sc.nextInt();

        Product p = new Product(name, price);
        Order o = new Order();

        o.placeOrder(p, qty);
    }
}

```

Enter Product Name: air conditioner
 Enter Price: 88809
 Enter Quantity: 5

 Product Name : air conditioner
 Price : 88809.0
 Quantity : 5
 Total Amount : 444045.0

9. Student Result Analyzer using Arrays of Objects

Design a Student class containing roll number, name, and marks in three subjects. Store multiple student objects in an array and implement methods to calculate total, average, grade, and class topper. Also, construct suitable test cases to validate grading and topper identification. BTL: BL3 (Apply)

CODE:

```

import java.util.Scanner;

class Student {
    int rollNo;

    String name;

    int m1, m2, m3;

    int total;

    double average;

    char grade;

    void getData(Scanner sc) {
        System.out.print("Enter Roll Number: ");

        rollNo = sc.nextInt();

        sc.nextLine();

        System.out.print("Enter Name: ");

        name = sc.nextLine();

        System.out.print("Enter Marks in 3 Subjects: ");

        m1 = sc.nextInt();
    }
}

```

```

        m2 = sc.nextInt();

        m3 = sc.nextInt();
    }

    void calculate() {
        total = m1 + m2 + m3;

        average = total / 3.0;

        if (average >= 90)
            grade = 'A';
        else if (average >= 75)
            grade = 'B';
        else if (average >= 50)
            grade = 'C';
        else
            grade = 'F';
    }

    void display() {
        System.out.println("\nRoll No : " + rollNo);
        System.out.println("Name : " + name);
        System.out.println("Total : " + total);
        System.out.println("Average : " + average);
        System.out.println("Grade : " + grade);
    }
}

public class StudentResult {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Number of Students: ");

        int n = sc.nextInt();

        Student[] s = new Student[n];

        for (int i = 0; i < n; i++) {
            s[i] = new Student();

            System.out.println("\nEnter Details of Student " + (i + 1));

            s[i].getData(sc);

            s[i].calculate();
        }
    }
}

```

```

    }

    int max = s[0].total;

    int topper = 0;

    for (int i = 1; i < n; i++) {

        if (s[i].total > max) {

            max = s[i].total;

            topper = i;

        }

    }

    System.out.println("\nStudent Details");

    for (int i = 0; i < n; i++) {

        s[i].display();

    }

    System.out.println("\nTopper: " + s[topper].name);

    System.out.println("Total Marks: " + s[topper].total);

    sc.close();

}

}

```

OUTPUT:

The screenshot shows a Java IDE with a code editor on the left and an output console on the right. The code in the editor is a Java program that takes marks for three subjects, calculates the total, average, and grade for each student, and identifies the topper. The output console shows the program's execution, including prompts for marks and the resulting student details for three students.

```

public class StudentResult {
    public static void main(String[] args) {
        for (int i = 0; i < n; i++) {
            s[i] = new Student();
            System.out.println("\nEnter Details of Student " + (i + 1));
            s[i].getData(sc);
            s[i].calculate();
        }

        int max = s[0].total;
        int topper = 0;

        for (int i = 1; i < n; i++) {
            if (s[i].total > max) {
                max = s[i].total;
                topper = i;
            }
        }

        System.out.println("\nStudent Details");
        for (int i = 0; i < n; i++) {
            s[i].display();
        }

        System.out.println("\nTopper: " + s[topper].name);
        System.out.println("Total Marks: " + s[topper].total);

        sc.close();
    }
}

```

Enter Marks in 3 Subjects: 9
9
9

Student Details

Roll No : 99
Name : ammu
Total : 27
Average : 9.0
Grade : F

Roll No : 98
Name : guru
Total : 27
Average : 9.0
Grade : F

Roll No : 9
Name : 9
Total : 27
Average : 9.0
Grade : F

Topper: ammu
Total Marks: 27

10. Smart Home Device Controller using Hierarchical Inheritance

Develop a smart home system with a base class Device and derived classes Light, Fan, and AirConditioner. Implement methods to turn devices on/off and display power consumption. Use polymorphism to control devices through a common reference. Also, construct suitable test cases to validate device operations and power calculations. BTL: BL4 (Analyze)

CODE:

```
import java.util.Scanner;

// Base Class

class Device {

    void displayPower() {

        System.out.println("Power Consumption");

    }

}

// Light Class

class Light extends Device {

    void displayPower() {

        System.out.println("Light Power Consumption: 20 Watts");

    }

}

// Fan Class

class Fan extends Device {

    void displayPower() {

        System.out.println("Fan Power Consumption: 75 Watts");

    }

}

// AirConditioner Class

class AirConditioner extends Device {

    void displayPower() {

        System.out.println("Air Conditioner Power Consumption: 1500 Watts");

    }

}

// Main Class

public class SmartHome {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("1. Light");

        System.out.println("2. Fan");

        System.out.println("3. Air Conditioner");

        System.out.print("Enter Your Choice: ");

        int choice = sc.nextInt();

        Device d;

        switch (choice) {
```



```

        case 1:
            d = new Light();
            break;
        case 2:
            d = new Fan();
            break;
        case 3:
            d = new AirConditioner();
            break;
        default:
            System.out.println("Invalid Choice");
            sc.close();
            return;
    }
    d.displayPower();
    sc.close();
}
}

```

OUTPUT:

<pre> public class SmartHome { public static void main(String[] args) { Scanner sc = new Scanner(System.in); Device d; switch (choice) { case 1: d = new Light(); break; case 2: d = new Fan(); break; case 3: d = new AirConditioner(); break; default: System.out.println("Invalid Choice"); sc.close(); return; } d.displayPower(); sc.close(); } } </pre>	<pre> 1. Light 2. Fan 3. Air Conditioner Enter Your Choice: 3 Air Conditioner Power Consumption: 1500 Watts </pre>
---	--